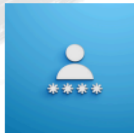
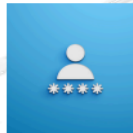


GROUP PROJECT

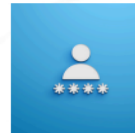
TEAM MEMBERS



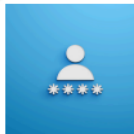
Aditya Kumar Sharma
(Team leader)
ID:20261651006



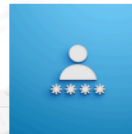
Sonam Raj
ID:20261651086



Aditi Choudhary
ID:20261651003



Sujit Kumar
ID:20261651089



Guru Prashad Panda
ID:20261651039

➤ OLD PROJECT

I am continuing the previous webpage by editing and adding new content to it.

[Events](#)[Registration](#)[Categories](#)[Contact Us](#)



IIIT Vadodara

Events

Upcoming Events

- Campus Hackathon**
 - Date: 10 October 2026
 - Time: 9:00 AM - 9:00 PM
 - Venue: Innovation Lab
- Tech Fest**
 - Date: 15 October 2026
 - Time: 10:00 AM - 5:00 PM
 - Venue: College Auditorium
- Coding Competition**
 - Date: 20 October 2026
 - Time: 11:00 AM - 2:00 PM
 - Venue: Computer Lab
- Sports Day**
 - Date: 25 October 2026
 - Time: 8:00 AM - 4:00 PM
 - Venue: College Ground
- Cultural Fest**
 - Date: 30 October 2026
 - Time: 5:00 PM - 9:00 PM
 - Venue: College Auditorium

Category List

Event Name	Category	Meeting Day
Campus Hackathon	Technical	Sunday
Tech Fest	Technical	Wednesday
Coding Competition	Technical	Saturday
Sports Day	Sports	Thursday
Cultural Fest	Cultural	Friday

Quick Information

Press tab to move through the links and form fields.
Use [events](#) to jump to the events section.
IIIT Vadodara hosts various college events.

Event Registration

Interested candidates can register for any upcoming event.

Candidate Name:

Email:

Phone Number:

Select Event:

Event Date:

Location:

Additional Information:

[Register Now](#)

Frequently Asked Questions

- Can I register for more than one event?
- How can I contact the event team?

Contact Us

For event registration and general queries, contact us.

Name	Aditya Kumar
Email	adityakumar6655829@gmail.com
Phone	+91 9876543210
Organization	IIIT Vadodara
Purpose	Event Registration and Queries

Disclaimer: This website is created for the 2026 event.
Team Leader: Aditya Kumar Sharma
Team Members: Suijt Kumar, Aditi Choudhary, Sonam Raj, Guru Prasad Panda
© 2026 IIIT Vadodara Event

[Events](#)[Registration](#)[Categories](#)[Contact Us](#)[FAQ](#)

Contact: Adityakumar6655829@gmail.com
Phone: +91 9876543210
IIIT Vadodara

➤ Striped table:

CSS style:

```
/* Even rows */
tr:nth-child(even) td {
    background-color: #7e7e58;
}
```

Output:

With CSS

Category List

Event Name	Category	Meeting Day
Campus Hackathon	Technical	Sunday
Tech Fest	Technical	Wednesday
Coding Competition	Technical	Saturday
Sports Day	Sports	Thursday
Cultural Fest	Cultural	Friday



Candidate Name:

Email:**Phone Number:****Select Event:****Event Date:****Location:**

Additional Information:

[Register Now](#)

[Register Now](#)

➤ CSS code-Styled form:

```
/* email and date */
input[type="email"],
input[type="date"] {
  border-color:  #4b6cb7;
  background-color:  #f8f8ff;
}
```

```
/* FORM */
input,
select,
textarea {
  padding: 8px;
  border: 2px solid  gray;
}
```

```
/* form keyboard focus */
input:focus,
select:focus,
textarea:focus {
  outline: 3px solid  yellow;
  border-color: var(--accent);
}
```



OUTPUTS:

Candidate Name:

Enter your name

Email:

Enter your email

Phone Number:

Enter your phone number

Select Event:

-- Select an Event -- ▾

Event Date:

dd-mm-yyyy



Location:

Enter event location

Additional Information:

Enter any additional information

Register Now

➤ The W3C CSS validator with zero errors:



The W3C CSS Validation Service

W3C CSS Validator results for style.css (CSS level 3 + SVG)

Jump to: [Validated CSS](#)

W3C CSS Validator results for style.css (CSS level 3 + SVG)

Congratulations! No Error Found.

This document validates as [CSS level 3 + SVG](#) !

To show your readers that you've taken the care to create an interoperable Web page, you may display this icon on any page that validates. Here is the XHTML you could use to add this icon to your Web page:



```
<p>
  <a href="https://jigsaw.w3.org/css-validator/check/referer">
    
    </a>
  </p>
```

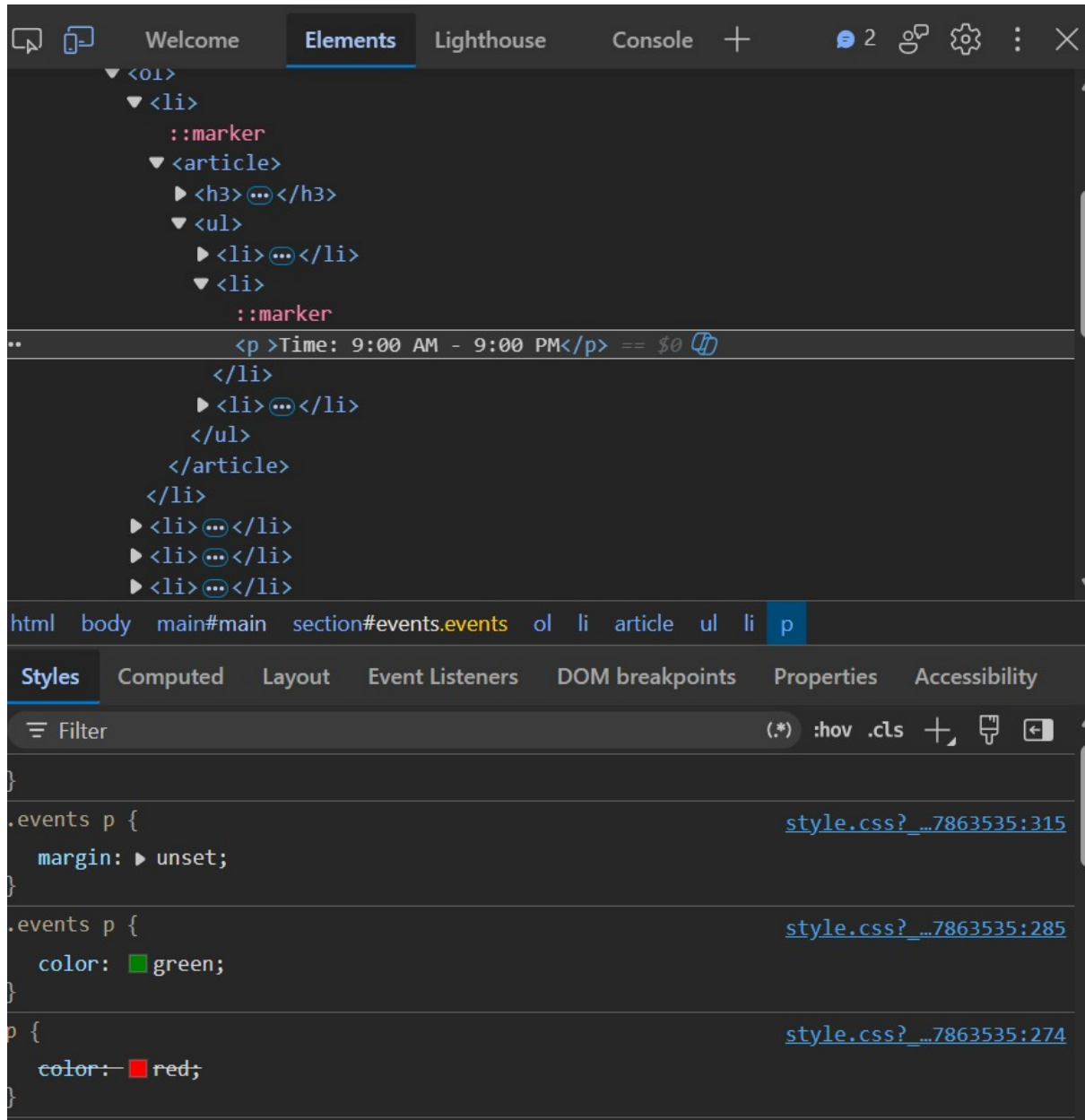


```
<p>
  <a href="https://jigsaw.w3.org/css-validator/check/referer">
    
    </a>
  </p>
```

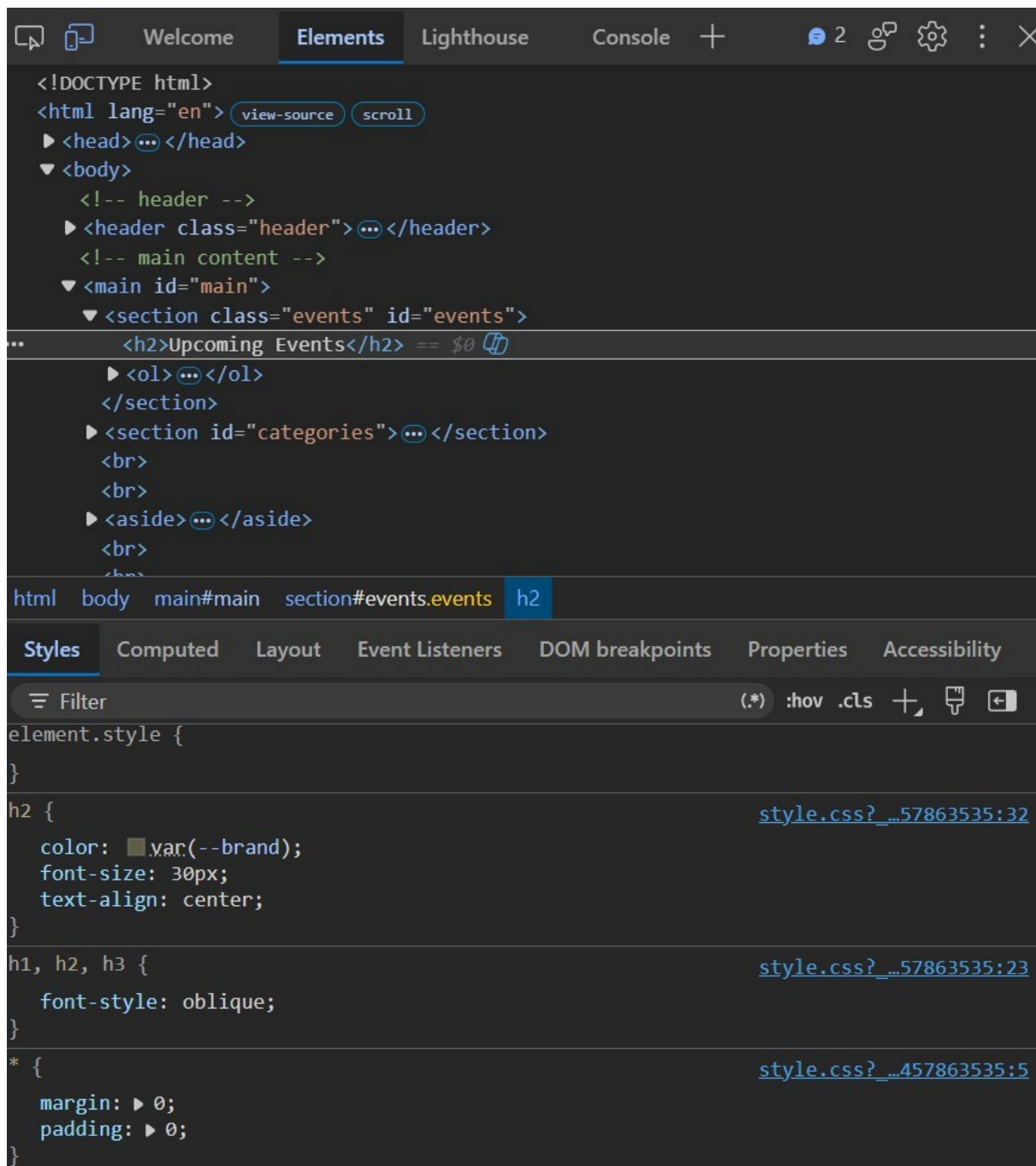


DEVTOOLS:

DevTools Styles Pane: Losing Rule Crossed Out



DevTools Styles Pane: Applied Rule



ANSWER OF THE QUESTIONS:

1.Score the ten selectors from B1 as (ids, classes, tags) and name the winner.

Answer: The specificity of each selector is measured as (IDs, Classes/Pseudo-classes, Tags/Elements). The scores are: $p = (0, 0, 1)$; $.events = (0, 1, 0)$; $nav .events = (0, 1, 1)$; $\#site-header = (1, 0, 0)$; $\#site-header .events = (1, 1, 0)$; $a: hover = (0, 1, 1)$; $ul li a = (0, 0, 3)$; $input[type="email"] = (0, 1, 1)$; $* = (0, 0, 0)$; $:not(.active) = (0, 1, 0)$. Winner: $\#site-header .events$, with $(1, 1, 0)$, because an ID is present and it also contains a class.

2. In B2, which rule won, and which reason decided it — source order or specificity?

Answer: For the given `<p class="events">` inside `<nav id="site-header">`, all three rules match. The winning rule is `#site-header .events { color: blue; }`. The deciding factor is specificity, not source order: $p = (0,0,1)$, $.events = (0,1,0)$, while $\#site-header .events = (1,1,0)$. The ID-based selector therefore wins.

CODE:

```
P {  
  Color :red;  
}  
.events p {  
  Color: green;  
}
```

3. Why does !important break the cascade, and what should you do instead?

Answer: !important changes the normal cascade priority, so an important declaration can override ordinary declarations even when those declarations have higher specificity. Frequent use makes CSS difficult to maintain because normal overrides and component-level rules stop behaving predictably. Recommended approach: avoid !important in normal styling. Use clear cascade layers, appropriate selector specificity, source order, and component structure to resolve conflicts.

Reserve !important for exceptional cases such as controlled utility overrides or unavoidable third-party CSS conflicts.

```
P {  
  Color :red;  
}  
.events p {  
  Color: green;  
}  
Color: orange !important
```

4. Descendant (A B) vs child (A > B) — a one-line difference, with an example from your page.

Answer: A B selects B at any depth inside A, whereas A > B selects only B that is a direct child of A.

```
Nav>a{  
  Margin:5px;  
  Padding:8px;  
}
```

5. Adjacent (A + B) vs general sibling (A ~ B) — a one-line difference.

Answer: A + B selects the immediately following sibling B, while A ~ B selects any following sibling B that shares the same parent.

```
H2+p{
Font-weight: bold;
}
H2-p {
padding-left:10px;
}
```

6. Name one property that inherited from body and one that did not; how did inherit / initial / unset help you?

Answer: A common inherited property is color: a child normally receives the body's color when it has no own color declaration. A common non-inherited property is margin: it does not automatically pass from parent to child. inherit explicitly takes the parent value; initial resets the property to its CSS initial value; unset acts as inherit for inherited properties and as initial for non-inherited properties.

Code:

```
body {
font-family: Arial, sans-serif;
}
input, select, text area, button {
font-family: inherit;
}
button {
line-height: initial;
}
.events p {
margin: unset;
}
```

7. What does the universal selector * score, and why is a * reset so easily overridden?

Answer: The universal selector * has specificity (0, 0, 0). A reset such as * { margin: 0; } is therefore easy to override because almost any more specific selector—such as an element, class, attribute, pseudo-class, or ID selector—has a higher specificity. A later rule with equal specificity can also win by source order.

Code:

```
* {  
margin: 0;  
padding: 0;  
}  
.events {  
padding: 30px;  
margin: 20px;  
}
```

8. :is() vs :where() — the one important difference, and when you would use :where().

Answer: The key difference is specificity: :is() takes the specificity of its most specific argument, while :where() always contributes (0, 0, 0) specificity. Use :where() when you want reusable base styles that remain easy for component or utility rules to override.

Code:

```
:is(h1,h2,h3){  
Font-style: oblique;  
}  
:where(.card, .panel){  
Margin:10px;  
}
```

9. What does @layer give you that fighting specificity does not?

Answer: @layer provides an explicit, predictable cascade order between groups of CSS rules. This lets developers decide which layer should take precedence without increasing selector specificity or creating long selector chains. It is especially useful for organizing reset, base, components, and utility styles.

Code:

```
@layer reset, base, components;
```

```
@layer reset {
```

```
* { margin: 0; padding: 0; }
```

```
}
```

```
@layer components {
```

```
.events { padding: 30px; }
```

```
}
```

10. Paste three of your selectors with their specificity scores — are they as low as they could be?

Answer: Three selectors from B1 are shown below. $p = (0,0,1)$, $.events = (0,1,0)$, and $\#site-header .events = (1,1,0)$. The first two are already minimal for targeting an element type and a class. The third is intentionally more specific because it targets an `.events` element within a specific ID container; if that container restriction is not required, `.events` would be a lower-specificity alternative. In general, selectors should be kept as simple and low-specificity as the design allows.

Code:

```
nav > a {
```

```
}
```

```
.events p {
```

```
}
```

```
\#site-header .events {
```